



The Design Process

HCI : Chapter 2

NORSHADILA BINTI AHMAD BADELA
JABATAN TEKNOLOGI MAKLUMAT & KOMUNKASI,
POLITEKNIK MERSING



LEARNING OUTCOME



- ▶ Understand the software engineering and basics design of interaction
- ▶ Understand design rules for interactive system

2.1 Understand the software engineering and basics design of interaction

Interaction Design

- Is the professional discipline that defines the behavior of interactive products and how products communicate their functionality to the people who use them.
- Good interaction design makes products ranging from computer software to clock radios to cars more useful, usable and desirable.

Software Engineering and Interaction Design Basics

- Software Engineering and design process **relate** to interactive system design
- The Software Engineering life cycle aims to structure design in order to increase the reliability of the design process

Is it true that Interaction Design is...

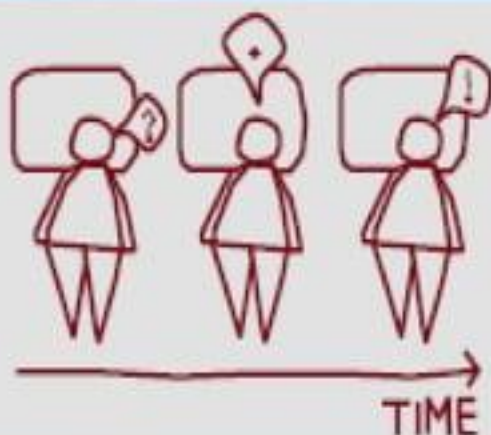
concerned with
making
something cool
or interesting
happen on
the screen



concerned with
the person in
front of the
screen, with
understanding
and
communication



concerned with
the conversation
back and forth
between people
and machines
across time



What is Interaction Design?

- Interaction design is **about creating interventions** in often complex situations using technology of many kinds including PC software, the web and physical devices
- Good interaction design makes products ranging from computer software to clock radios to cars more useful, usable and desirable.

Four basic activities in Interaction Design:

1. Identifying needs and establishing requirements
2. Developing alternative designs
3. Building interactive versions of the designs
4. Evaluating designs

► Three key characteristics permeate these four activities:

1. Focus on users early in design and evaluation of artefact
2. Identify, document and agree specific usability and user experience goals
3. Iteration is inevitable.
 - Designers never get it right first time

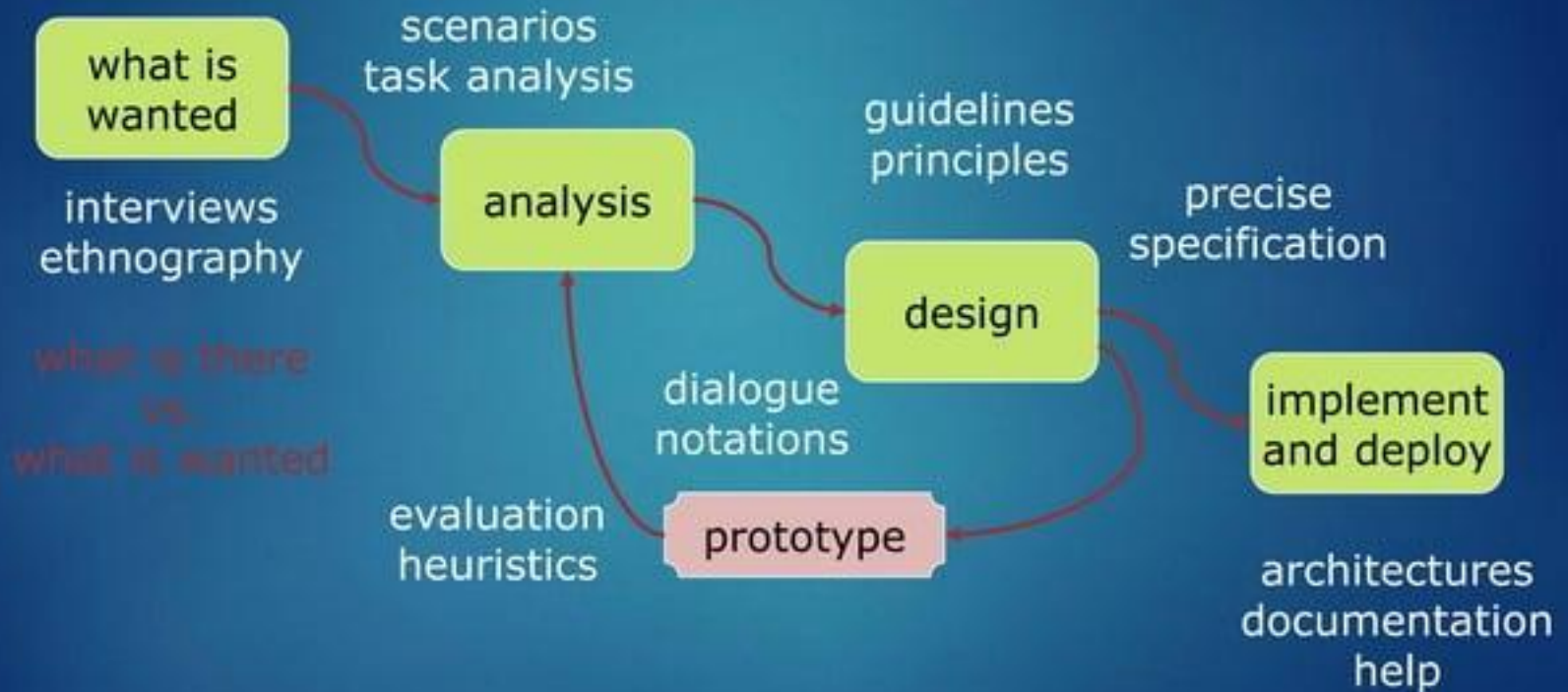
Software Development Process That Related To Interactive System Design

- ▶ Software engineering is the discipline for understanding the software design process, or life cycle.
- ▶ Designing for usability occurs at all stages of the life cycle, not as a single isolated activity.

Software Lifecycle Models

- ▶ Briefly relate design to software engineering ideas
 - ▶ Familiar to most
- ▶ Show how activities are related to each other
- ▶ Lifecycle models are:
 - ▶ management tools
 - ▶ simplified versions of reality
- ▶ Many lifecycle models exist, for example:
 - ▶ from software engineering:
 - ▶ — waterfall, spiral, JAD/RAD, Microsoft
 - ▶ from HCI:
 - ▶ — Star, usability engineering

The process of design



i. Requirements – what is wanted

- ▶ The first stage is establishing what exactly is needed.
- ▶ As a precursor to this it is usually necessary to find out what is currently happening.
- ▶ For example, how do people currently watch movies?
- ▶ There are a number of techniques used for this in HCI: interviewing people, videotaping them, looking at the documents and objects that they work with, observing them directly.

ii. Analysis

- ▶ The results of observation and interview need to be ordered in some way to bring out key issues and communicate with later stages of design.
- ▶ In this step, we will look at scenarios, rich stories of interaction, which can be used in conjunction with a method like task analysis or on their own to record and make vivid (jelas) actual interaction.
- ▶ These techniques can be used both to represent the situation as it is and also the desired situation.

iii. Design

- ▶ This is all about design, but there is a central stage when you move from what you want and how to do it.
- ▶ There are numerous (pelbagai) rules, guidelines and design principles that can be used.
- ▶ We need to record our design choices in some way and there are various notations and methods to do this, including those used to record the existing situation.
- ▶ At this stage, input from theoretical work is most helpful, including cognitive models, organizational issues and understanding communication.

iv. Iteration and prototyping

- ▶ Humans are complex and we cannot expect to get designs right first time.
- ▶ We therefore need to evaluate a design to see how well it is working and where there can be improvements.
- ▶ Some forms of evaluation can be done using the design on paper, but it is hard to get real feedback without trying it out.
- ▶ Most user interface design therefore involves some form of prototyping, producing early versions of systems to try out with real users.

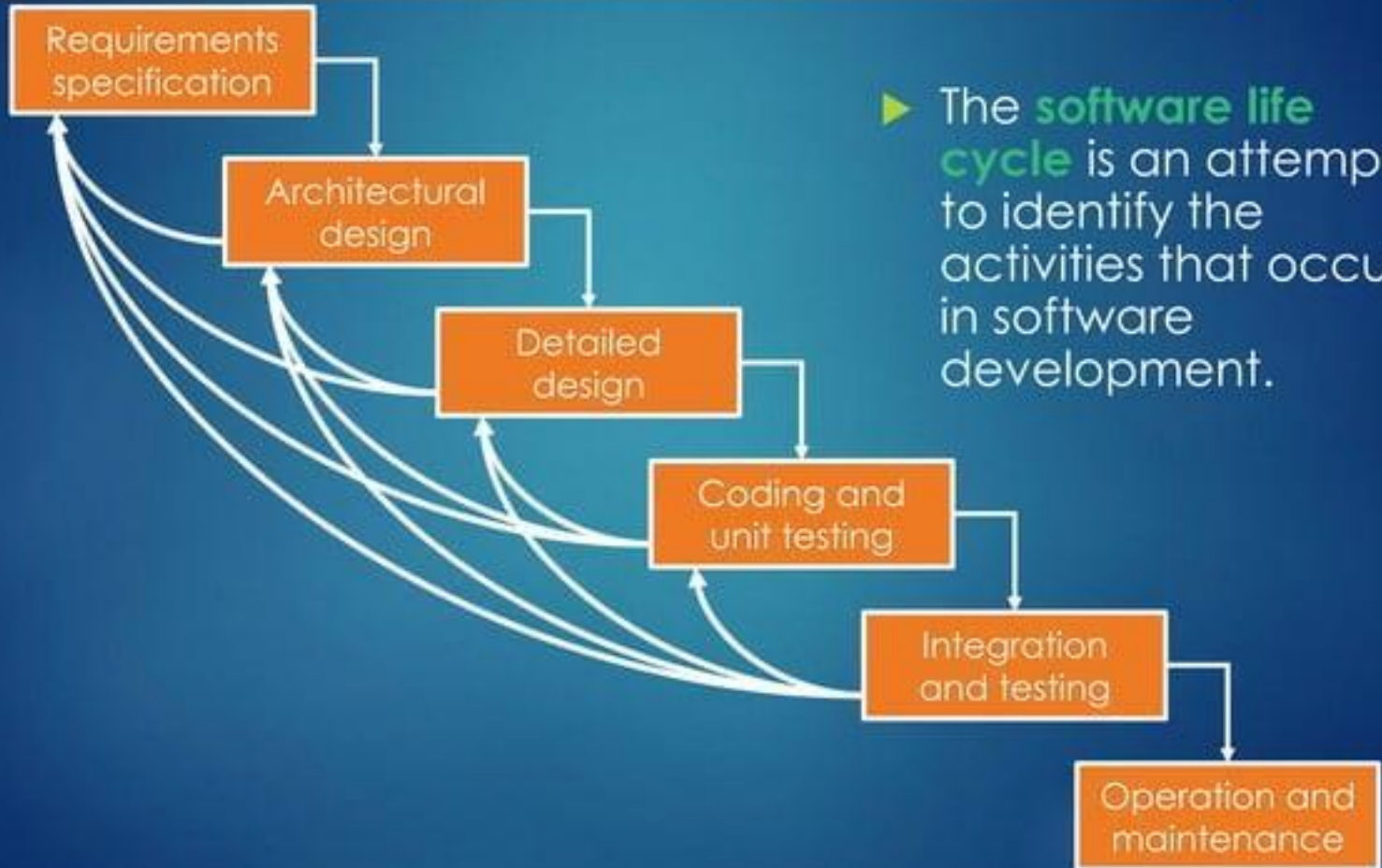


v. Implementation and deployment

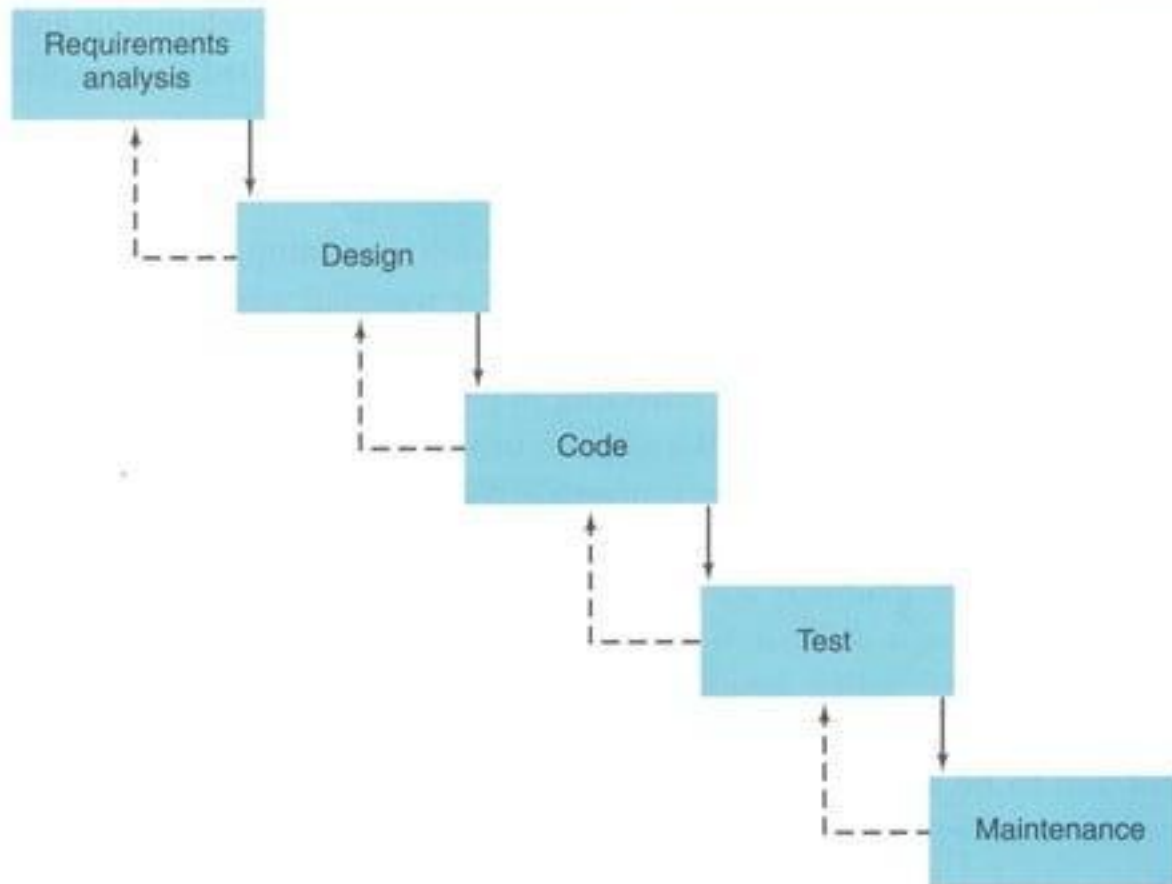
- ▶ Finally, when we are happy with our design, we need to create it and deploy it.
- ▶ This will involve writing code, perhaps making hardware, writing documentation and manuals – everything that goes into a real system that can be given to others.

The life cycle for interactive systems

- The **software life cycle** is an attempt to identify the activities that occur in software development.



The waterfall lifecycle model



Activities in Software Life Cycle : Waterfall Model

1. Requirements specification

- ▶ Involves eliciting information from the customer about the work environment in which the final product will function
- ▶ From customer's perspective, they must be formulated in language suitable for implementation
- ▶ Initially expressed in the native language of the customer
- ▶ Key of successful development:
 - ▶ • The transformation from the expressive but relatively ambiguous natural language of requirements to the more precise but less expressive executable language

2. Architectural Design

- ▶ How the system provides the services expected from it
- ▶ First activity is a high level decomposition of the system into components that can be either be brought in from existing software products or be developed from scratch independently
- ▶ Architectural Design provides a decomposition of the system description that allows for isolated development of separate components which will later be integrated

3. Detailed Design

- ▶ Is a refinement of the component description provided by the architectural design
- ▶ The language used for detailed design must allow some analysis of the design in order to assess its properties.
- ▶ It is important to keep track of the design options considered, the eventual decisions that were made and the reasons why.

4. Coding and unit testing

- ▶ After coding, the component can be tested to verify that it performs correctly, according to some test criteria that were determined in earlier activities

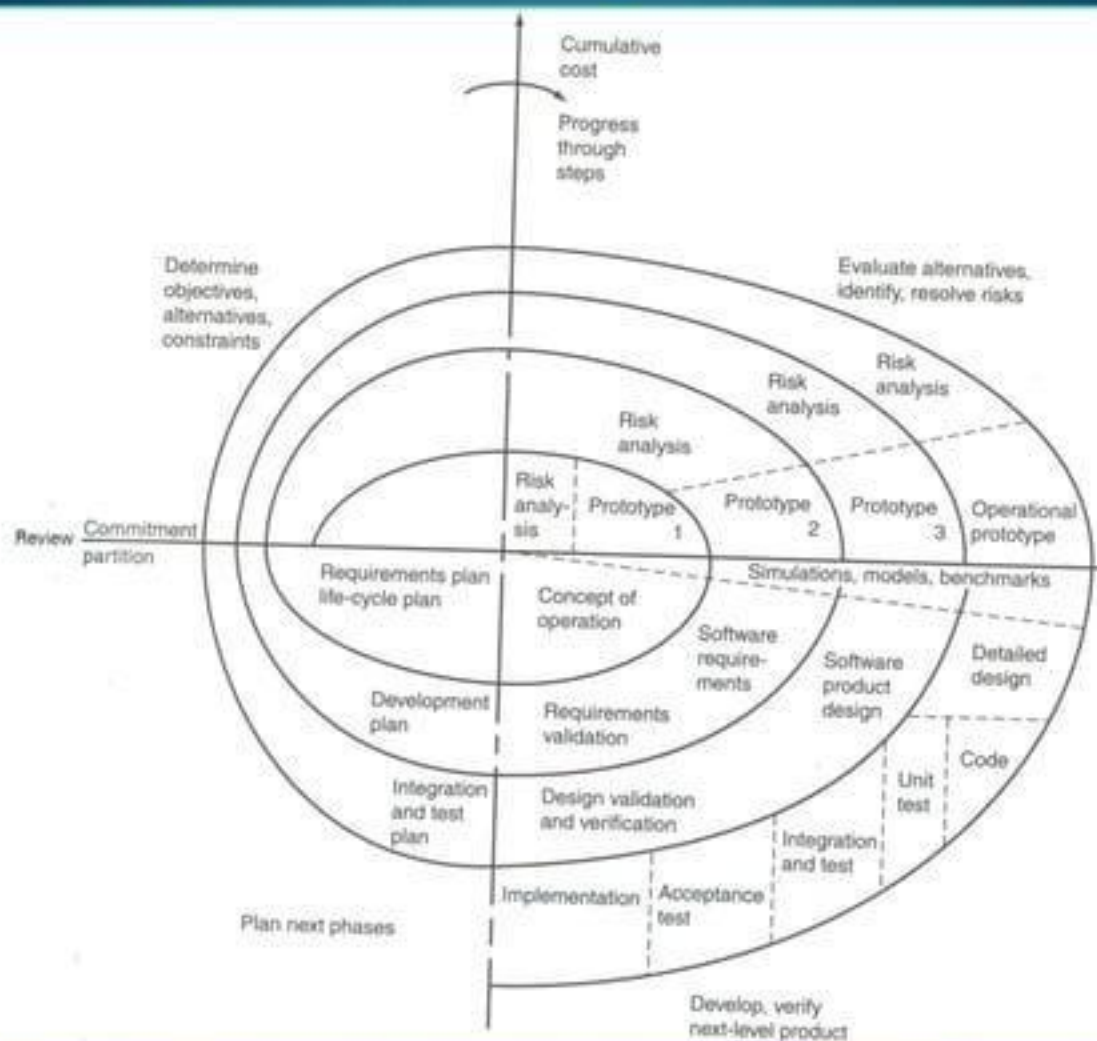
5. Integration and testing

- ▶ To ensure correct behavior and acceptable use of any shared resources
- ▶ To perform some acceptance testing with customers to ensure that the system meets their requirements
- ▶ It is only after acceptance of the integrated system that the product is finally released to the customer
- ▶ To certify the final system according to requirements imposed by some outside authority

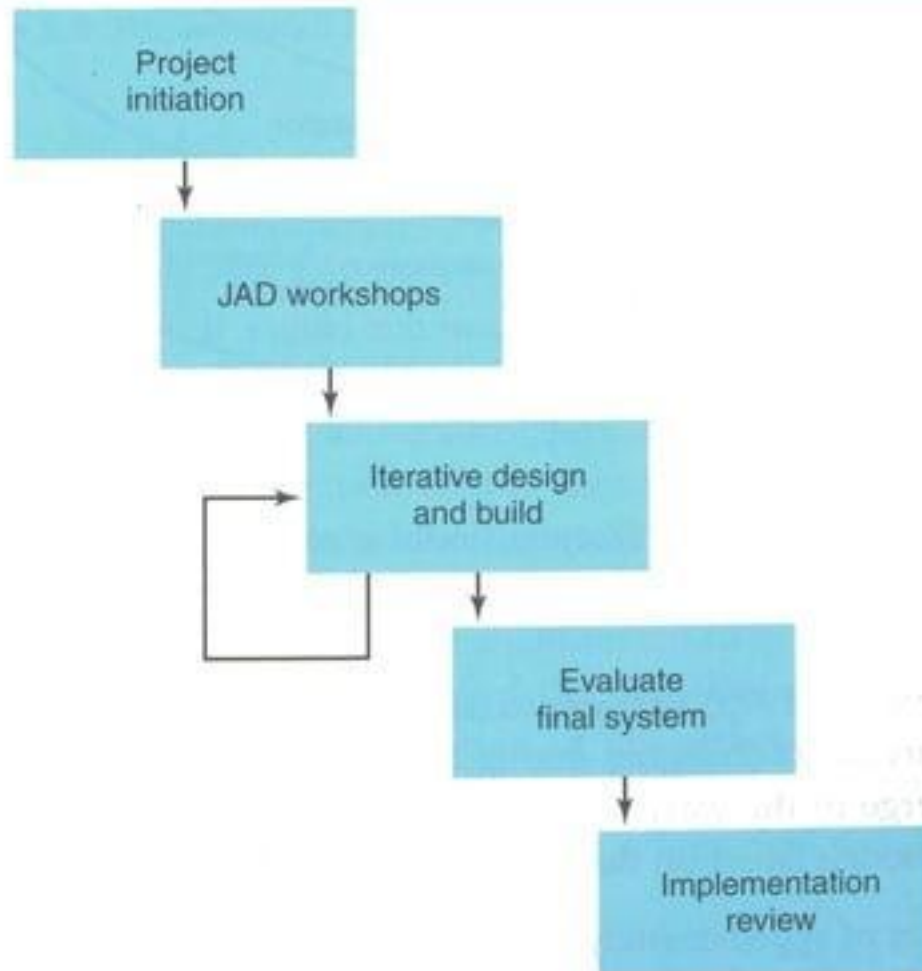
6. Maintenance

- ▶ After product release, all work on the system is considered under the category of maintenance, until such time as a new version of the product demands a total redesign or the product is phased out entirely
- ▶ Majority of the lifetime of a product is spent in the maintenance activity
- ▶ Maintenance involves the correction of errors in the system which are discovered after release and the revision of the system services to satisfy requirements that were not realized during previous development

The spiral lifecycle model

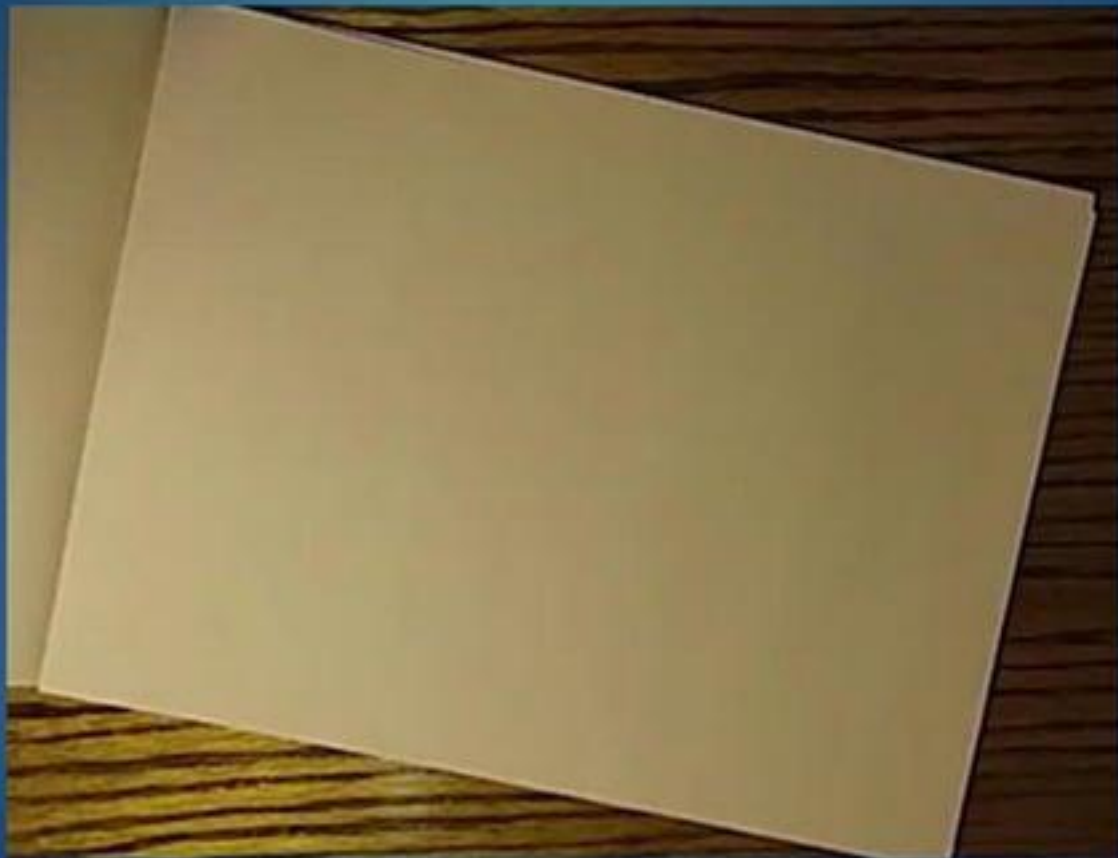


The RAD lifecycle model



The following video is on how to design card deck.

Based on the idea of that video, list down your own idea on creating 1 traditional games (congak, batu Seremban, dam aji, teng teng, lompat getah)





Two main concept in Design Process

i. Usability Engineering

ii. Iterative Design

a) Usability engineering

- ▶ The emphasis for usability (kebolehgunaan) engineering is in knowing exactly what criteria will be used to judge a product for its usability.
- ▶ The ultimate test of a product's usability is based on measurements of users' experience with it.
- ▶ In relation to the software life cycle, one of the important features of usability engineering is the inclusion (kemasukan) of a usability specification.

a) Usability engineering

Usability specification

- ▶ usability attribute/principle
- ▶ measuring concept
- ▶ measuring method
- ▶ now level/ worst case/ planned level/ best case

Attribute: Backward recoverability

Measuring concept:	Undo an erroneous programming sequence
Measuring method:	Number of explicit user actions to undo current program
Now level:	No current product allows such an undo
Worst case:	As many actions as it takes to program-in mistake
Planned level:	A maximum of two explicit user actions
Best case:	One explicit cancel action

b) Iterative design

- ▶ Practices work to incorporate crucial customer feedback early in the design process to inform critical decisions which affect usability
- ▶ Requirements for an interactive system cannot be completely specified from the beginning of the life cycle.
- ▶ The only way to be sure about some features of the potential design is to build them and test them out on real users.
- ▶ The design can then be modified to correct any false assumptions (andaian) that were revealed in the testing.
- ▶ This is the essence of iterative design, a purposeful design process which tries to overcome the inherent problems (masalah yg wujud) of incomplete requirements specification by cycling through several designs.

b) Iterative design

- ▶ On the technical side, iterative design is described by the use of prototypes, artifacts that simulate or animate some but not all features of the intended system.
- ▶ There are 3 main approaches to prototyping:

Throw-away

Incremental

Evolutionary

- ▶ Management Issues : time, planning, non-functional features, contracts
- ▶ Technique prototype:
 - ▶ **Storyboard** : need not be computer-based can be animated
 - ▶ **Limited functionality simulations** : some part of system functionality provided by designers tools like HyperCard are common for these
- ▶ Wizard of Oz technique
 - ▶ Warning about iterative design
 - ▶ design inertia – early bad decisions stay bad
 - ▶ diagnosing real usability problems in prototypes....
 - ▶ and not just the symptoms

Advantages of Iterative model

In iterative model we can only create a high-level design of the application before we actually begin to build the product and define the design solution for the entire product. Later on we can design and built a skeleton version of that, and then evolved the design based on what had been built.

In iterative model we are building and improving the product step by step. Hence we can track the defects at early stages. This avoids the downward flow of the defects.

In iterative model we can get the reliable user feedback. When presenting sketches and blueprints of the product to users for their feedback, we are effectively asking them to imagine how the product will work.

In iterative model less time is spent on documenting and more time is given for designing.

Disadvantages of Iterative model

Each phase of an iteration is rigid with no overlaps

Costly system architecture or design issues may arise because not all requirements are gathered up front for the entire lifecycle

Testing prototypes to choose among alternatives



<http://www.combimouse.com>

Paper Prototyping – Example I

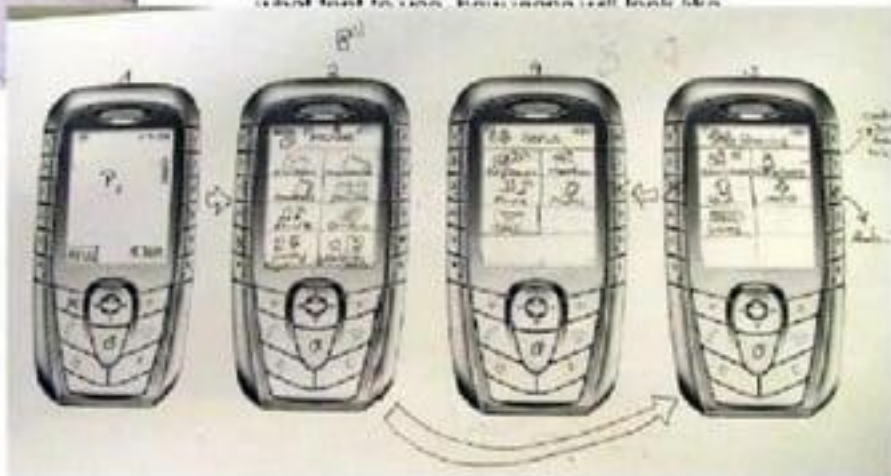


Sketches & Storyboards

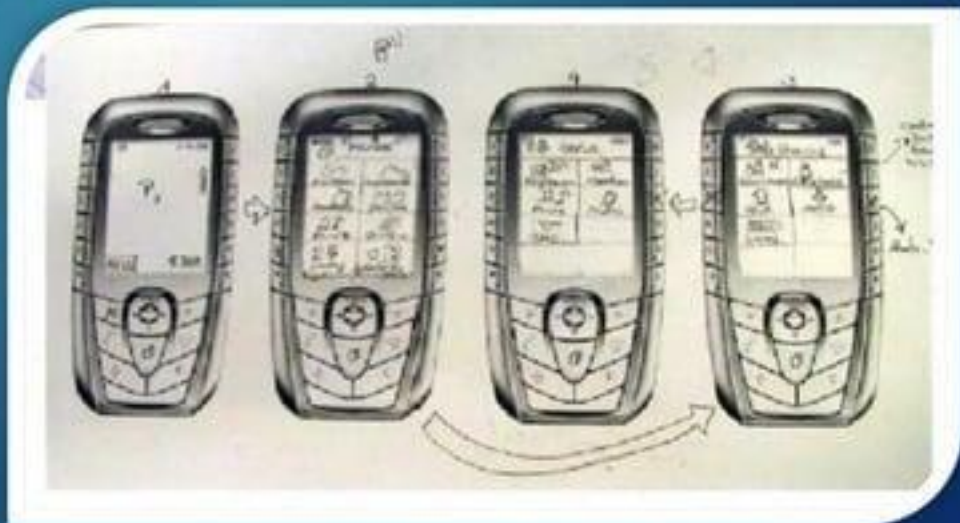


- Storyboards as for movies
 - A picture for each key scene
- Sketch out the application
 - Key screens
 - Main interaction
 - Important transitions
- Helps to communicate and validate ideas
 - Easy to try out different option, e.g. document base vs. application based
- Ignore details, e.g.

what font to use, how many will look like



- Based on the idea of creating 1 traditional games (congkak, batu Seremban, dam aji, teng teng, lompat getah) that you have analyze previously, build a prototype of online traditional games.



2.2 Understand design rules for interactive system

- One of the central problems that must be solved in a design process is how to provide designers with the ability to determine the usability consequences of their design decisions.
- We need **Design Rules**, which are rules, a designer can follow in order to increase the usability of the eventual software product.
- Design rules for interactive systems can be supported by :
 - i. psychological,
 - ii. cognitive,
 - iii. ergonomic,
 - iv. sociological,
 - v. economic or
 - vi. computational theory.

Design rules for Interactive Systems



- ▶ Design rules are mechanisms for restricting the space of design options, preventing a designer from pursuing design options that would be likely to lead to an unusable system
- ▶ To provide designers with the ability to determine the usability consequences of their design decision
- ▶ Can be supported by psychological, cognitive, ergonomic, sociological, economic or computational theory

Types of design rules

- ▶ **principles**

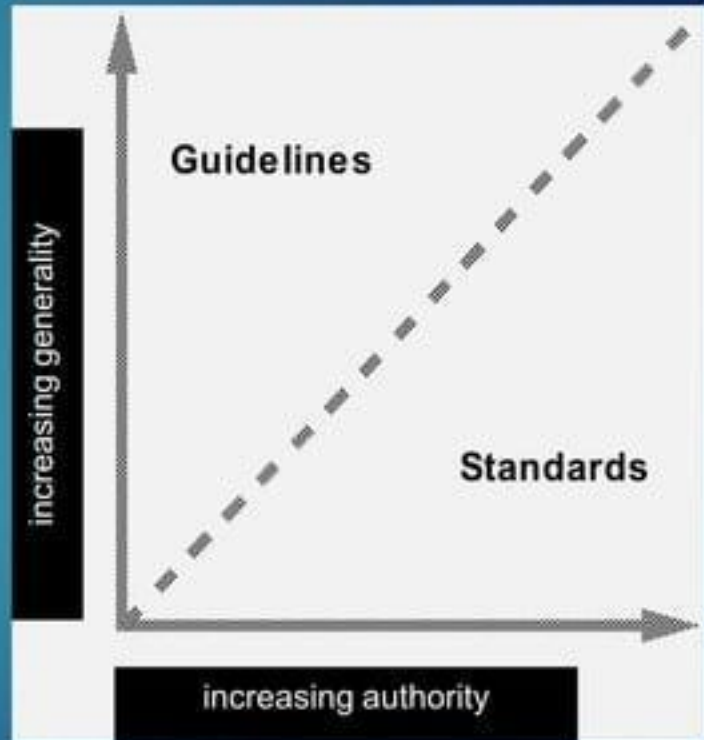
- ▶ abstract design rules
- ▶ low authority
- ▶ high generality

- ▶ **standards**

- ▶ specific design rules
- ▶ high authority
- ▶ limited application

- ▶ **guidelines**

- ▶ lower authority
- ▶ more general application



Principles to support usability in interactive systems design

- The most abstract design rules are general principles, which can be applied to the design of an interactive system in order to promote its usability.
- Principles can provide the repeatability which paradigms in themselves cannot provide.
- Three main categories of that principles:
 - a) **Learnability**
 - b) **Flexibility**
 - c) **Robustness**

a) *Learnability*

- Concerns the features of the interactive system that allow novice users **to understand how to use** it initially and then how to attain a maximal level of performance
 - Predictability
 - Synthesizability
 - Familiarity
 - Generalizability
 - Consistency

Summary of principles affecting learnability

Principle	Definition	Related Principles
Predictability	Support for the user to determine the effect of future action based on past interaction history	Operation visibility
Synthesizability	Support for the user to assess the effect of past operations on the current state	Immediate/eventual honesty
Familiarity	The extent to which a user's knowledge and experience in other real-world or computer-based domains can be applied when interacting with a new system	Guess ability, affordance
Generalizability	Support for the user to extend knowledge of specific interaction within and across applications to other similar situations	-
Consistency	Likeness in input-output behavior arising from similar situations or similar task objectives	-

Principles of Learnability (1 / 2)

- Predictability

- determining effect of future actions based on past interaction history
- operation visibility



- Synthesizability

- ability of the user to assess the effect of past operations on the current state
- the user should see the changes of an operation
- immediate vs. eventual feedback



```
C:\WINDOWS\system32\cmd.exe
C:\>cd test.txt test
C:\test>dir
Volume in drive C has no label.
Volume Serial Number is FCB2-5668

Directory of C:\

05.05.2007  12:36                8 installBshq.txt
               1 File(s)
               8 Dir(s)  14,052,261,888 bytes free

C:\test>cd test
C:\test>dir
Volume in drive C has no label.
Volume Serial Number is FCB2-5668

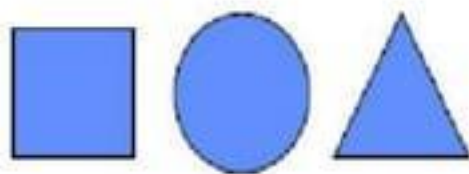
Directory of C:\test

09.11.2007  16:54                8 test.txt
               1 File(s)
               8 Dir(s)  14,052,261,888 bytes free

C:\test>
```

Principles of Learnability (2 / 2)

- Familiarity
 - how prior knowledge applies to new system
 - affordance (guessability)
- Generalizability
 - extending specific interaction knowledge to new situations
- Consistency
 - likeness in input/output behavior arising from similar situations or task objectives



b) Flexibility

- ▶ Flexibility refers to the multiplicity of ways in which the **end-user and the system exchange information**
 - ▶ Dialog initiative
 - ▶ Multi-threading
 - ▶ Task migratability
 - ▶ Substitutivity
 - ▶ Customizability

Principles Of Flexibility

Principle	Definition	Related Principles
Dialog Initiative	Allowing the user freedom from artificial constraints on the input dialog imposed by the system	System/user pre-emptiveness
Multi-threading	Ability of the system to support user interaction pertaining to more than one task at a time	Concurrent vs. interleaving, multi-modality
Task migratability	The ability to pass control for the execution of a given task so that it becomes either internalized by the user or the system or shared between them	-
Substitutivity	Allowing equivalent values of input and output to be arbitrarily substituted for each other	Representation multiplicity, equal opportunity
Customizability	Modifiability of the user interface by the user or the system	Adaptivity, adaptability

Principles of Flexibility (1 / 6)

- Ways in which the user and the system exchange information
- Dialogue initiative
 - freedom from system imposed constraints on input dialogue
 - user preemptiveness: user initiates dialog
 - system preemptiveness: system initiates dialog



user preemptiveness

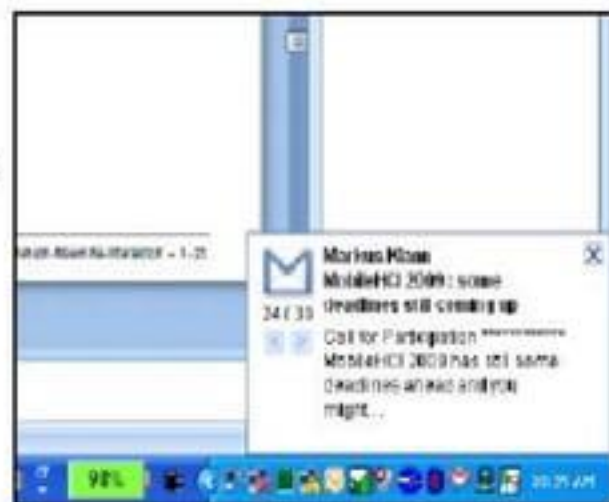


system preemptiveness

Principles of Flexibility (2 / 6)

- Multithreading

- ability of system to support user interaction for several tasks at a time
- concurrent multimodality: simultaneous communication of information pertaining to separate tasks
 - » multi-model dialog
 - » editing text and beep (incoming mail) at the same time
- interleaving multimodality: permits temporal overlap between separate tasks, dialog is restricted to a single task
 - » window system, window = task
 - » modal dialogs
 - » interaction with just one window at a given time



Principles of Flexibility (3 / 6)

- Task migratability
 - passing responsibility for task execution between user and system
 - example: spell checking

Task migratability



Principles of Flexibility (4 / 6)

- Substitutivity
 - allowing equivalent values of input and output to be substituted for each other
 - representation multiplicity



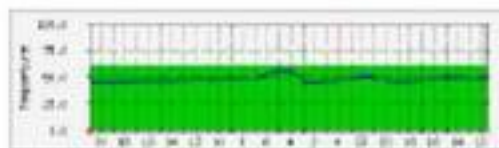
Actual / Print Size

Width: 35.555 inches

Height: 14.222 inches

Resolution: 72 cm

- equal opportunity: blurs the distinction between input and output



	A	B	C	D
1				
2	Summand 1	1	2	1
3	Summand 2	2	2	2
4	Summand 3	3	3	3
5	Total sum	6	7	6
6				

Principles of Flexibility (5 / 6)

- Customizability
 - modifiability of the user interface by the user (adaptability) or system (adaptivity)
 - **adaptability**: users ability to adjust the form of input and output
 - **adaptivity**: automatic customization of the user interface by the system



Principles of Flexibility (6 / 6)

- Customizability
 - modifiability of the user interface by the user (adaptability) or system (adaptivity)
 - adaptability: users ability to adjust the form of input and output
 - adaptivity: automatic customization of the user interface by the system



c) *Robustness*

- ▶ Covers features that support the **successful achievement** and assessment of the goals
 - ▶ Observability
 - ▶ Recoverability
 - ▶ Responsiveness
 - ▶ Task conformance

Summary of principles affecting robustness

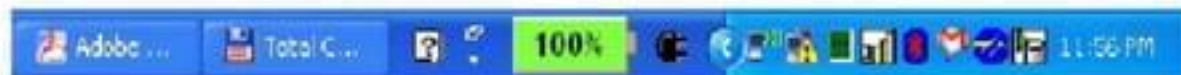
Principle	Definition	Related Principles
Observability	Ability of the user to evaluate the internal state of the system from its perceivable	Browsability, static/dynamic defaults, reachability, persistence, operation visibility
Recoverability	Ability of the user to take corrective action once an error has been recognizes	Reachability, forward/backward recovery, commensurate effort
Responsiveness	How the user perceives the rate of communications with the system	Stability
Task conformance	The degree to which the system services support all of the tasks the user wishes to perform and in the way that the user understand them	Task completeness, task adequacy

Principles of Robustness (1 / 2)

the level of support provided to the user in determining successful achievement and assessment of goal-directed behavior

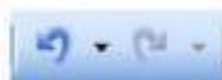
→ Level of support provided to the user in determining successful achievement and assessment of goal-directed behavior

- Observability

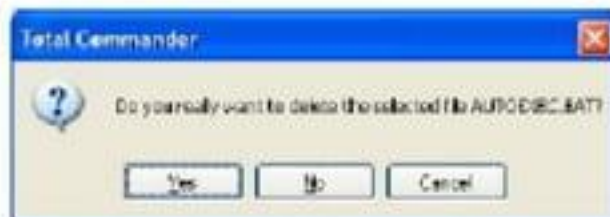


- ability of the user to evaluate the internal state of the system from its perceivable representation

- Recoverability



- ability of the user to correct a recognized error
 - reachability (states): forward (redo) / backward (undo) recovery
 - commensurate effort (more effort / steps for deleting a file than for moving it)



Principles of Robustness (2 / 2)

- Task conformance
 - degree to which system services support all of the user's tasks
 - task completeness; task adequacy
- Responsiveness
 - how the user perceives the rate of communication with the system
 - preferred: short durations and instantaneous responses ($< 100\text{ms}$)
 - stability and indication of response time



PowerPoint is saving work My Documents\work\lmu\lehrauftrag2009\lectur...

Letterboxing: Please wait.
This may take a while.

Standards

- ▶ set by national or international bodies to ensure compliance by a large community of designers standards
- ▶ require sound underlying theory and slowly changing technology
- ▶ hardware standards more common than software high authority and low level of detail
- ▶ **ISO 9241**
 - ▶ ISO 9241 is a multi-part standard covering a number of aspects for people working with computers.
 - ▶ Originally known as *Ergonomic Requirements for Office Work with Visual Display Terminals (VDT)*.
 - ▶ Revised in 2006, re-titled as *Ergonomics of Human-System Interaction*.

The revised multipart standard is numbered in series as follows:

- 100 series: Software ergonomics
- 200 series: Human system interaction processes
- 300 series: Displays and display related hardware
- 400 series: Physical input devices - ergonomics principles
- 500 series: Workplace ergonomics
- 600 series: Environment ergonomics
- 700 series: Application domains - Control rooms
- 900 series: Tactile and haptic interactions

Guidelines

- ▶ more suggestive and general
- ▶ many textbooks and reports full of guidelines
- ▶ abstract guidelines (principles) applicable during early life cycle activities
- ▶ detailed guidelines (style guides) applicable during later life cycle activities
- ▶ understanding justification for guidelines aids in resolving conflicts
- ▶ **Golden rules and heuristics**
 - ▶ Provide a convenient and succinct summary of the key principles of interface design.
 - ▶ 'Broad brush' design rules – may not always applicable to every situation.
 - ▶ There are many sets of rules :
 - ▶ Nielsen's 10 Heuristics (Chapter 3)
 - ▶ Norman's 7 Principles (chapter 1)
 - ▶ Shneiderman's 8 Golden Rules

Schneiderman's Eight Golden Rules of interface design

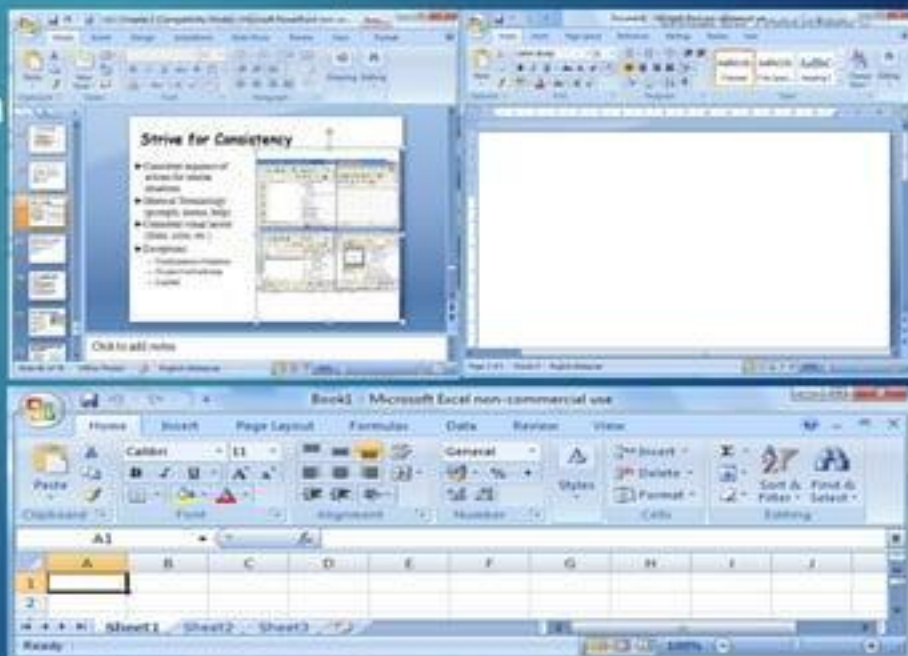
- ▶ A number of advocates of user-centered design have presented sets of '**golden rules**' or heuristics.
- ▶ **Shneiderman's eight golden rules** provide a convenient and succinct summary of the key principles of interface design.
- ▶ They are intended to be used during design but can also be applied, like Nielsen's heuristics, to the evaluation of systems.
- ▶ Like those principles, they are not applicable to every eventuality and need to be interpreted for each new situation.

Schneiderman's Eight Golden Rules

- 1. Strive for consistency*
- 2. Enable frequent users to use shortcuts*
- 3. Offer informative feedback*
- 4. Design dialogs to yield closure*
- 5. Offer error prevention and simple error handling*
- 6. Permit easy reversal of actions*
- 7. Support internal locus of control*
- 8. Reduce short-term memory load*

1. STRIVE FOR CONSISTENCY

- ▶ **Strive for consistency** in action sequences, layout, terminology, command use and so on.
- ▶ Identical Terminology (prompts, menus, help)
- ▶ Consistent visual layout (fonts, color, etc.)
- ▶ Exceptions:
 - Confirmation of deletion
 - No password echoing
 - Limited



2. *ENABLE FREQUENT USERS TO USE SHORTCUTS*

- ▶ **Enable frequent users to use shortcuts**, such as abbreviations, special key sequences and macros, to perform regular, familiar actions more quickly.

3. *OFFER INFORMATIVE FEEDBACK*

- ▶ **Offer informative feedback** for every user action, at a level appropriate to the magnitude of the action.
- ▶ For every user action, the system should provide feedback

4. *DESIGN DIALOGS TO YIELD CLOSURE*

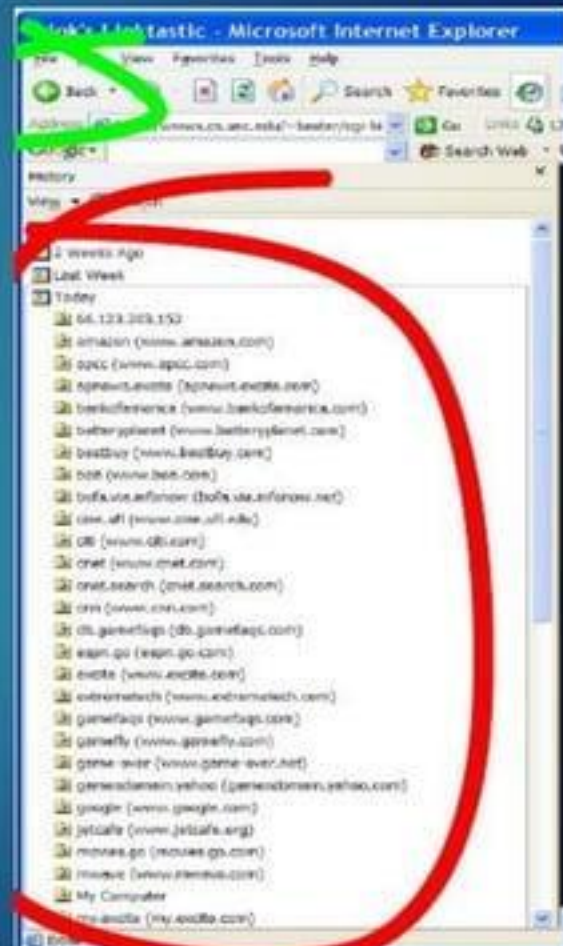
- ▶ **Design dialogs to yield closure** so that the user knows when they have completed a task.
- ▶ Action sequences should have a beginning, middle, and end.
- ▶ Feedback provides sense of accomplishment.
- ▶ Ex. Purchasing items via internet has a clearly defined step-by-step process

5. OFFER ERROR PREVENTION AND SIMPLE ERROR HANDLING

- ▶ ***Offer Error Prevention and Simple Error Handling*** so that, ideally, users are prevented from making mistakes and, if they do, they are offered clear and informative instructions to enable them to recover.

6. PERMIT EASY REVERSAL OF ACTIONS

- ▶ **Permit Easy Reversal of Actions** in order to relieve anxiety and encourage exploration, since the user knows that he can always return to the previous state.



7. *SUPPORT INTERNAL LOCUS OF CONTROL*

- ▶ **Support Internal Locus of Control** so that user is in control of the system, which responds to his actions.
- ▶ Experienced operators want to feel in control.
 - User is in charge of the interface
 - Interface rapidly responds to the user
- ▶ Lack of control builds anxiety and dissatisfaction.
 - Surprising interface actions
 - Tedious actions
 - Difficulty in obtaining necessary ability
 - Difficulty in producing action
- ▶ **Good rules:** Avoid a causality, make users initiators rather than responders

8. *REDUCE SHORT-TERM MEMORY LOAD*

- ▶ **Reduce short term memory load** by keeping display simple, consolidating multiple page displays and providing time for learning action sequences.

2.1.6 Design rationale

- ▶ Design rationale is the information that explains why a computer system is the way it is, including its structural or architectural description and its functional or behavioral description.
- ▶ In this sense, design rationale does not fit squarely into the software life cycle described in this chapter as just another phase or box.
- ▶ Design rationale relates to an activity of both reflection (doing design rationale) and documentation (creating a design rationale) that occurs throughout the entire life cycle.

2.1.6 Design rational

Process-oriented design rational

Design space analysis

Psychological design rational

Design Rationale

- ▶ Design rationale is information that explains why a computer system is the way it is including :
 - ▶ its structural or architectural description
 - ▶ its functional or behavioral description
- ▶ Benefits of design rationale
 - ▶ communication throughout life cycle
 - ▶ reuse of design knowledge across products
 - ▶ enforces design discipline
 - ▶ presents arguments for design trade-offs
 - ▶ organizes potentially large design space
 - ▶ capturing contextual information

TYPES OF DESIGN RATIONALE

- ▶ **Process-oriented**

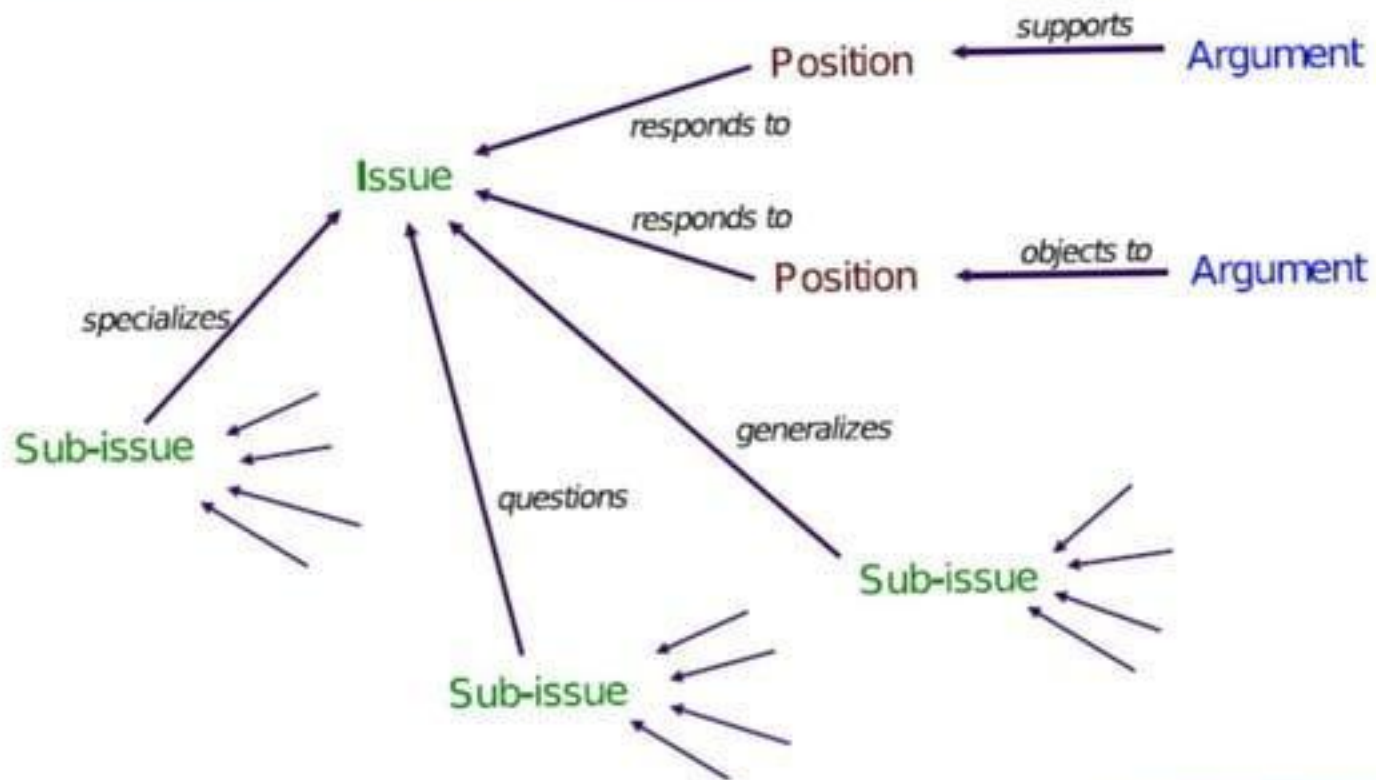
- ▶ preserves order of deliberation and decision-making

- ▶ **Structure-oriented (Design Space Analysis)**

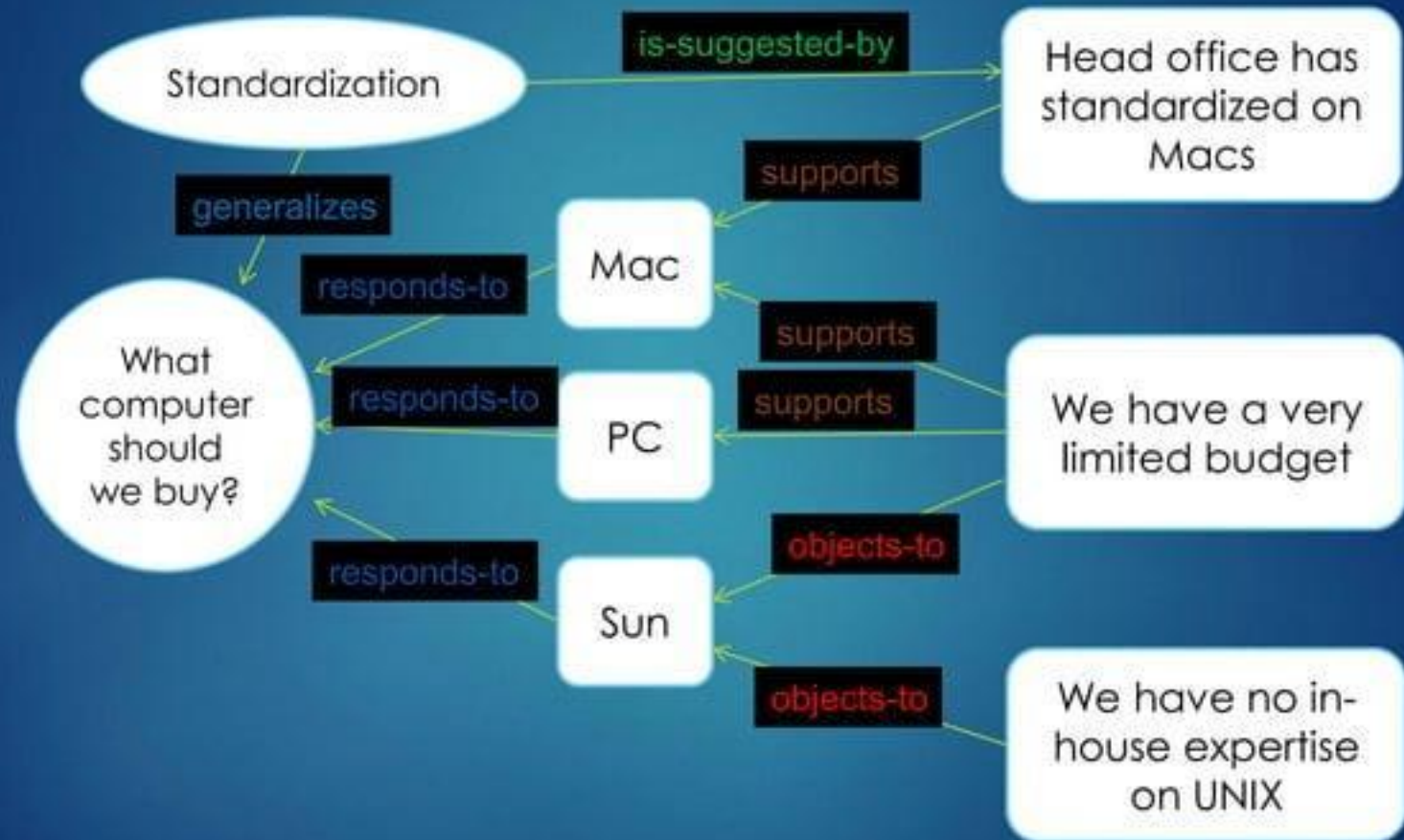
- ▶ emphasizes post hoc structuring of considered design alternatives

- ▶ **Psychological design rationale**

Structure of gIBIS



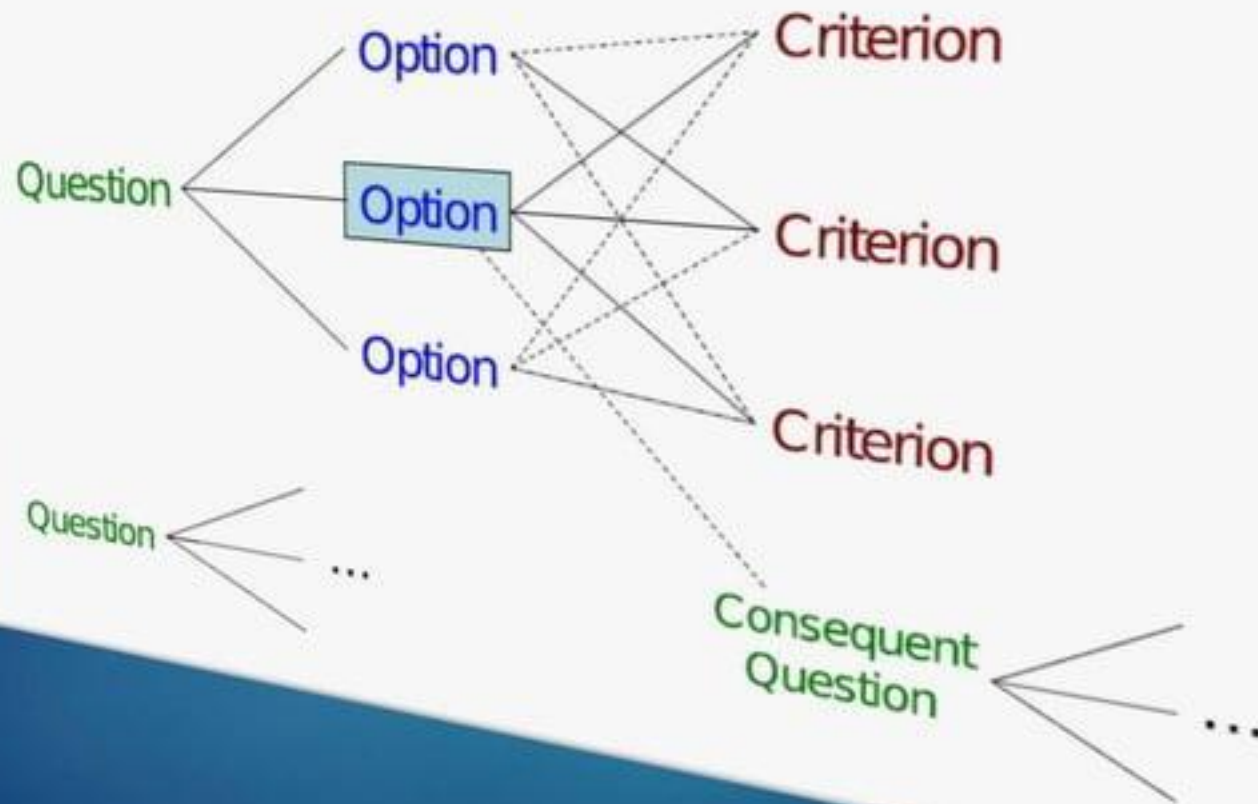
A gIBIS Discussion: Buying a Computer



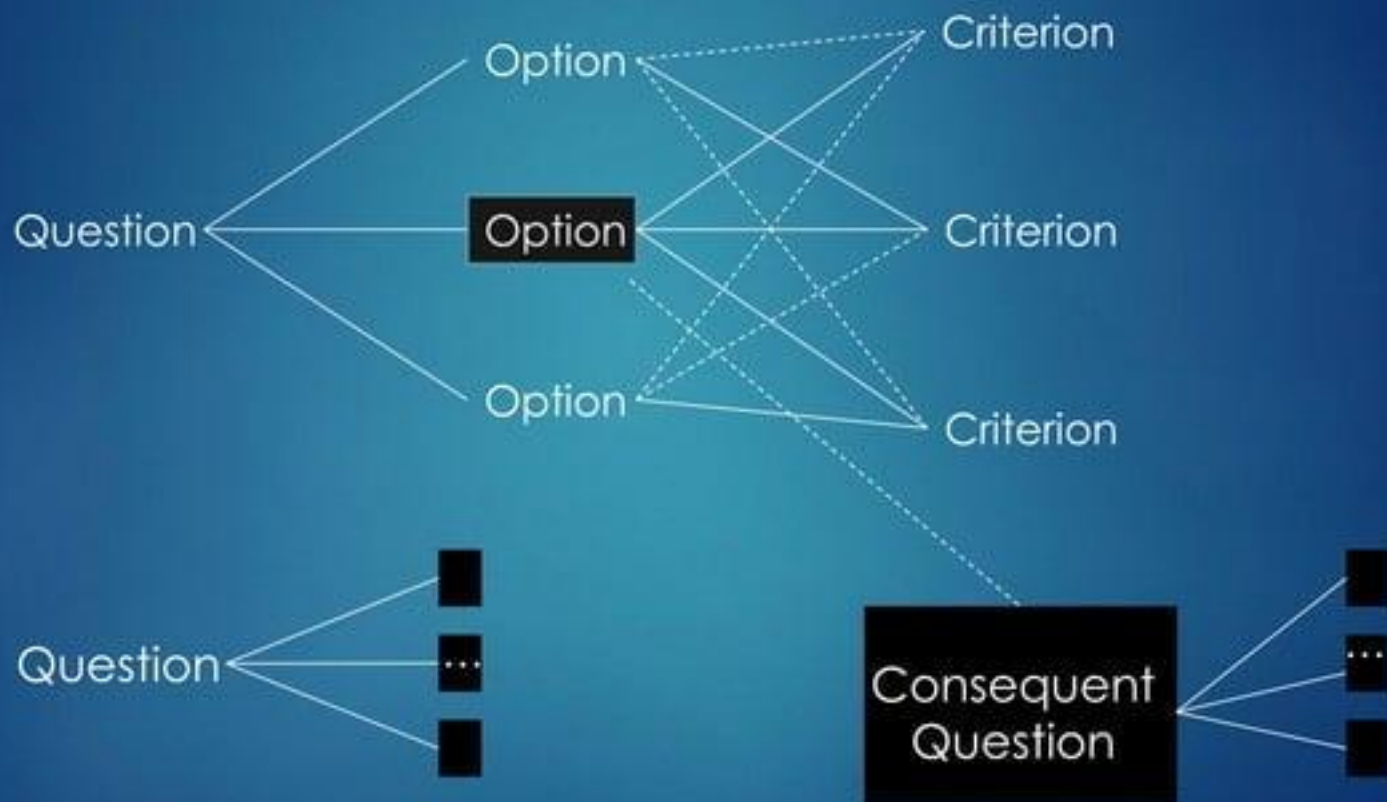
Design space analysis

- ▶ Design space is initially structured by a set of questions representing the major issues of the design
- ▶ Structured oriented
- ▶ QOC (Question Option Criterion)
- ▶ QOC – hierarchical structure:
- ▶ Questions (and sub-questions)
 - ▶ represent major issues of a design
- ▶ Options
 - ▶ provide alternative solutions to the question
- ▶ Criteria
 - ▶ the means to assess the options in order to make a choice
- ▶ DRL(Decision Representation Language) – similar to QOC with a larger language and more formal semantics

DESIGN SPACE ANALYSIS



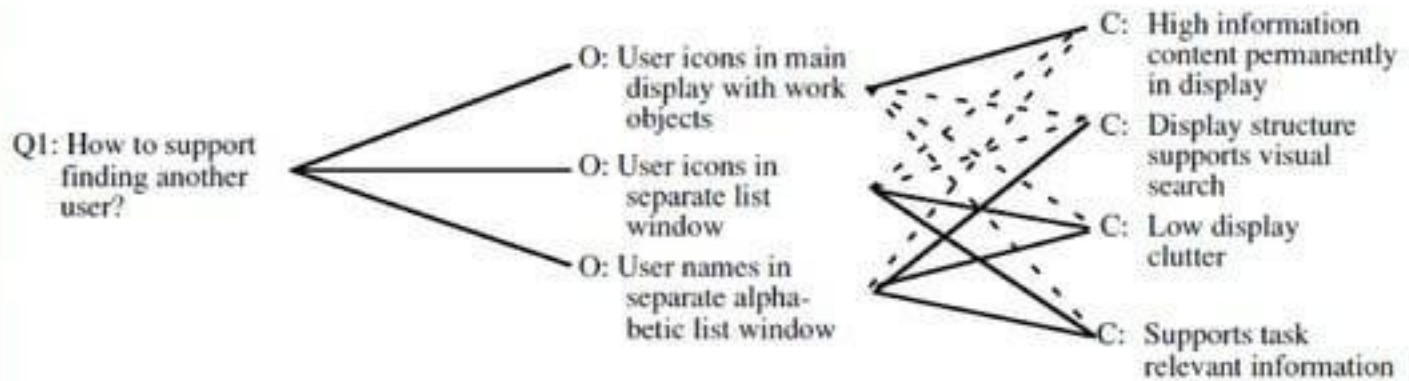
the QOC notation



QOC Example

- ▶ Goal:
 - ▶ Finding information about current objects of user is working on
- ▶ Questions
 - ▶ How to support finding another user?
 - ▶ How to show what work objects a person is using.
- ▶ How is the best interaction design to support it?

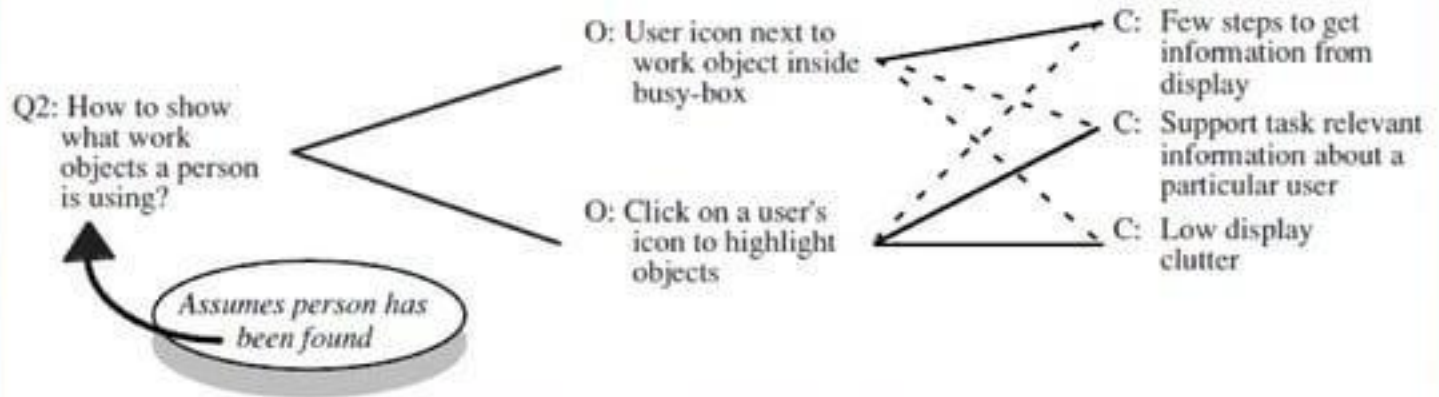
How to support Finding Another User?



Search



How to Show What Work Objects a Person is Using



PSYCHOLOGICAL DESIGN RATIONALE

- ▶ To support task-artifact cycle in which user tasks are affected by the systems they use.
- ▶ Aims to make explicit consequences of design for users
- ▶ Designers identify tasks system will support
- ▶ Scenarios are suggested to test task
- ▶ Users are observed on system
- ▶ Psychological claims of system made explicit
- ▶ Negative aspects of design can be used to improve next iteration of design

Example Of Psychological Design Rationale

- ▶ Designing a system to help programmers learn the Smalltalk OO programming language environment.
- ▶ Main task: support learning how Smalltalk works.
 - ▶ The programmer (user) will perform tasks that help her answer the questions:
- ▶ What can I do: what are the possible operations or functions that this programming environment allows?
- ▶ How does it work: what do the various functions do?
- ▶ How can I do this: once I know a particular operation I want to perform, how I go about programming it?
 - ▶ To address the question 'What can I do?':
- ▶ Designers describe a scenario whereby the novice programmer use the system for the first time.
- ▶ Provide demos to support user to investigate how Smallwork programs work.
- ▶ Observe user use the system (once the system running).
 - ▶ For example, the psychological claim on demo system, user learns by doing.
- ▶ Negative aspect, user may not learn anything from the demo (may not be interactive).
 - ▶ The negative aspect can be addressed in new version.

Benefit of interface design from design rationale

1. Provides a communication mechanism among members of a design team to avoid incorrect assumptions later
2. Accumulated knowledge can be reused which has similar needs
3. Forces the designer to deliberate more carefully about design decisions
4. The designer will determine which criteria for evaluating the options more important
5. The designer indicates all alternatives that have been investigated
6. Capturing the context in which a design decision is made will help later when new products are designed

Process-oriented design rationale

- ▶ Is based on Rittel's *issue-based information system* (IBIS)
- ▶ A style for representing design and planning dialog develop in 1970s.
- ▶ IBIS : is a hierarchical structure to a design rationale is created
- ▶ Issues : a root issue is identified which represents the main problem or question that the argument is addressing
- ▶ Position : various positions are put forth as potential resolutions for the root issue
- ▶ Argument : modify the relationship between positions and issues
- ▶ A graphical version of IBIS has been define which makes the structure of the design rationale more apparent visually in the form of a directed graph which can be directly edited by the creator of the design rationale

Discussion:

- ▶ How interface design can benefit from design rationale
- ▶ Which design rationale will produce the best result? Explain why?
- ▶ Can you combine all design rationale to produce the best result? Explain why?

Ways of improving interface design of given example

- ▶ Hypertext systems
- ▶ Programming environments
- ▶ ATM's
- ▶ Voice answering systems
- ▶ Mail systems

Hypertext systems

- ▶ The hypertext system was an interface designed to facilitate access to large amounts of text in electronic form.
- ▶ Typical applications include technical reference manuals, online documentation, and scientific journal articles converted into online form.
- ▶ The system ran on a standard workstation using multiple windows and various other graphical user interface features such as the ability for the user to click on a word in the text with the mouse to find more information about that word.

Continue...

- ▶ **Navigable Spaces**

- ▶ **Examples:**

- ▶ WWW and other hypertext systems

- ▶ Myst

- ▶ Museum exhibit in a set of physical rooms

- ▶ Set of applications in a suite, as with the PalmPilot or a network computer

- ▶ **Context:** The artifact contains a large amount of content -- too much to be reasonably presented in a single view. This content can be organized into distinct conceptual spaces or working surfaces which are semantically linked to each other, so that it is natural and meaningful to go from one to another.

- ▶ **Problem:** How can you present the content so that a user can explore it at their own pace, in a way which is comprehensible and engaging to the user?

- ▶ **Forces:**

- ▶ The user wants to know where they can (or should) go next, and how it's related to where they are now.

- ▶ The user wants to be able to choose where to go next.

- ▶ The user doesn't want to get lost.

- ▶ The concept of information spaces is a natural one for people to think about, both because it mirrors the real world and because the WWW is so commonly understood.

- ▶ It's delightful to explore a new place, where the user doesn't necessarily know what's "around the corner."

Continue...

- **Solution:** Create the illusion that the working surfaces are spaces, or places the user can "go" into and out of. Start out with at least one top-level or "home" space, to which the user can easily return ([Clear Entry Points](#)). In each space, clearly indicate how you get to the next space(s), such as by underlined text, buttons, images of doors, architectural features, etc. Use the spatial locations of these links to help the user remember where the links are. Provide a map of how the spaces are interconnected ([Map of Navigable Spaces](#)), preferably one that allows the user to go directly to the spaces represented on the map. Make sure that the user can easily retreat out of a space ([Go Back One Step](#)) or return to the home space ([Go Back to a Safe Place](#)).

The user will build a mental model of the content from the structure of the Navigable Spaces. Therefore, construct the spaces and their interconnections to mirror the model you want to present (which may not be the same as the actual underlying data structure). Chains, trees, and star patterns are common ways to structure Navigable Spaces (see illustration below); they are easy to understand, visualize, and navigate, and they can contain rich content.

Continue...

- ▶ **Resulting Context:** As pointed out above, Map of Navigable Spaces should be one of the first patterns you deal with, even if you explicitly choose not to use one; the same for Go Back One Step and Go Back to a Safe Place. To help show where the links are in the spaces, you can use Pointer Shows Affordance; to give additional information about where they go, use Short Description.
- ▶ People using the WWW tend to depend upon their browser's Interaction History (the links you've most recently visited, in chronological order) to get around. Not surprisingly, they also depend upon their Bookmarks to keep track of places they want to go back to. These two patterns might be especially important in any large or unbounded set of Navigable Spaces, particularly if a map is impractical.
- ▶ When you're dealing with power users, seriously consider the value of displaying more than one surface at a time, perhaps using Tiled Working Surfaces. It's often good to provide the user with the option of being in at least two or three spaces of their choice, especially if a user is likely to be jumping between spaces frequently. This does increase the user's cognitive load, though, so it may not be appropriate for simpler artifacts that require short learning curves.

Continue...

- ▶ **Notes:** With games, part of the fun is in figuring out where you are and where you can go next, so maps and obvious links would actually reduce the user's fun. In a way, the WWW is similar -- who could ever make a map of the WWW anyway? -- but, of course, not everyone uses it for fun.
- ▶ Notice that chains are structured similarly to Step-by-Step Instructions, trees to Hierarchical Set, and stars to Central Working Surface. All three of these archetypes have very strong, simple geometric properties; they probably warrant further exploration.

Programming Environments

- ▶ **Composed Command**
- ▶ **Examples:**
 - ▶ UNIX or DOS command-line interface
 - ▶ SQL
 - ▶ Spoken instructions to an interface made for visually impaired people
 - ▶ One human telling another to do something
 - ▶ Programming and scripting languages
- ▶ **Context:** The possible actions to be taken with the artifact can be expressed through commands, which can be composed from smaller parts, in a language-like syntax with precise and learnable rules; and the users are willing and able to learn that syntax.
- ▶ **Problem:** How can the artifact best present the actions that the user may take?
- ▶ **Forces:**
 - ▶ Expert users often find linguistic commands to be more efficient than visual representations or direct manipulation.
 - ▶ Sometimes the available actions cannot, or should not, be expressed graphically (perhaps because there is an intractably large set of them).
 - ▶ The visual channel may not be available at all to the artifact or to the user.
 - ▶ The artifact is able to provide feedback on the correctness and appropriateness of the given commands.

Continue...

- ▶ **Solution:** Provide a way for the user to directly enter the command, such as by speech or by typing it in. Feedback on the validity of the command, or its results, should be as immediate as is practical. The parts and syntax rules should be easy to learn, and should generate concise commands whose meaning is obvious. Offer a way to do auto-completion or a set of possible interpretations of partially-entered commands, especially if the user is unwilling or unable to learn the language -- but beware the expert user, who may find this irritating! Allow it to be turned off if necessary. **Resulting Context:** The possible actions divide neatly into environmental and object actions (see [Convenient Environment Actions](#) and [Localized Object Actions](#)); let the object actions accept "wildcards" in place of the object, to effect [Actions for Multiple Objects](#). [Forgiving Text Entry](#) lets the user give commands with a generous margin for error, which is necessary for a natural-language-style interface and pleasant for other kinds. [Reality Check](#) and [Progress Indicator](#) generally make dialogue-like actions easier for users to deal with.
- ▶ Most existing Composed Command systems provide some kind of [Interaction History](#). In the linear dialogue that this pattern imposes upon the user, a history can be invaluable; users often need to repeat previous commands, sometimes with minor changes, and they sometimes need to know what they've done recently. Composed Command also takes well to [Scripted Action Sequences](#) (also common in on-line implementations of this pattern), especially since users are already thinking in terms of grammar and composable parts.

ATM's

- ▶ **Step-by-Step Instructions**

- ▶ **Examples:**

- ▶ Wizards
- ▶ Installation instructions for all kinds of appliances, software, etc.
- ▶ Recipes
- ▶ Repair manuals
- ▶ Getting cash from an ATM

- ▶ **Context:** A user needs to perform a complex task, with limited time, knowledge, attention, or space. Alternatively, the nature of the task is step-by-step, and it's meaningless to show all the action possibilities at once. **Problem:** How can the artifact unfold the possible actions to the user in a way that does not overwhelm or confuse them, but instead guides them to a successful task completion?

- ▶ **Forces:**

- ▶ The user doesn't always want, or need, to understand all the details of what they are doing.
- ▶ A user presented with a bunch of possible actions, in no prescribed order, may not have any practical way to figure out what to do first, second, etc.
- ▶ A user who is afraid of doing something wrong may prefer that the actions they have to perform be explicitly spelled out for them.
- ▶ The whole task can't be performed entirely automatically, because it requires choices or information from the user.

Continue...

- **Solution:** Walk the user through the task one step at a time, giving very clear instructions at each step. Use visual similarities in all the steps, e.g. typography and layout, to maintain a rhythm throughout the task; make each step a focal point, both visually and in the user's "attention space." If information is needed from the user, ask for it in simple terms and with brevity; by keeping it short, you can better maintain the user's sense of flow through the whole step-by-step process. The task may branch like a flow chart, depending upon what information the user gives it, but the user doesn't necessarily need to know about all the available paths through the task. If it's not likely to confuse the user, show the steps as a Map of Navigable Spaces. If possible, allow the user to back out of the steps (Go Back One Step, Go Back to a Safe Place). If the sequence of steps as seen by the user is too long – more than ten steps, for example – try to break it up into manageable sub-sequences, so it doesn't get too tedious for the user. Make sure the sub-sequences relate to each other in a meaningful way, however, or the user may see it as gratuitous or annoying. Sometimes users may want to know more about what they're doing – Optional Detail On Demand gives you a way to present that extra information. Also, if a user has gone through a lot of steps, they have trouble remembering what they've done and why. At least provide a Progress Indicator if the number of steps grows beyond seven or eight, which is the average limit of short-term memory. If a lot of user interaction is necessary, such as for branching decisions, consider providing an Interaction History.

Continue...

- ▶ **Resulting Context:** Narrative is a good choice for presenting the task steps themselves; the use of natural language to describe what needs to be done is intuitively obvious, and puts a user at ease. Go Back One Step and Go Back to a Safe Place, along with a corresponding Forward control, can be used to move through an interactive task.
- ▶ To get information from the user, you can use a Form or its simpler component patterns, especially Choice from a Small Set and Forgiving Text Entry. Using Good Defaults with them allows the user to move smoothly past the points where extra data entry is unnecessary, again preserving the sense of flow. Finally, a small set of Convenient Environment Actions should give the user ways to cancel or suspend the task without having to back out of it one step at a time.
- ▶ **Notes:** Be aware that this pattern may irritate experienced users. If a user knows exactly what they need to do, and want to do it quickly, constraint to this step-by-step presentation can feel like a straitjacket! Also, if the task to be accomplished isn't inherently linear -- i.e. you don't really have to do one step first, another step second, etc. -- you might provide an alternative "random access" presentation of the possible actions, such as a Stack of Working Surfaces.

Voice Answering Systems

- ▶ Control Panel - The whole front of the phone.
Small Groups Of Related Things - The various groups of buttons: numbers, function buttons, etc.
Helper Posture - Only in your face while doing dialing, not too imposing on my desk.
Background Posture - The blinking 'Hold' indicator. (also Status Display)
Important Message - The ringer.
Remembered State - Redial button, stored numbers (bound to 'Quick Dial' buttons).
Go Back to a Safe Place - The 'Release' button, or the hook.
Composed Command - The dialing process (sometimes could be better supported by speech recognition)
Quick Access - 911, 411, *SP, etc.
User Preferences - The 'Quick Dial' buttons and the numbers they are bound to.
- ▶ Social Space - Conversations and multi-way conversations are supported by the medium the telephone connects to.
Narrative - Voice mail can be seen as a Narrative triggered by the user.
Optional Detail On Demand - Slide-out panel that shows more functionality, in more sophisticated phones.
Iconic Reference - Pictures indicating quick-dial buttons, like fire, police, pizza, etc.
Editable Collection - Voice mail 'unheard' and 'saved' messages.
Step-by-Step Instructions - Automated account information available from the phone company, credit card company, etc.

Voice answering systems

- ▶ **Important Message**
- ▶ **Examples:**
- ▶ Alert dialogs
- ▶ Ringing telephone
- ▶ Alarms of all sorts
- ▶ **Context:** While using the artifact, the user must be informed of something immediately. This often happens with *Status Display* and *Control Panel*, especially when they are used in life-critical situations, but could happen in other primary patterns as well.
- ▶ **Problem:** How should the artifact convey this information to the user?
- ▶ **Forces:**
- ▶ The user is probably paying attention to something else at the time the event happens.
- ▶ The user may be short of time or under stress, possibly as a result of the message itself, so they won't be in a good position to stop and think about how to react.
- ▶ Normal operation of the artifact after the event happens may be a bad idea.
- ▶ Too much repetition of a distracting cue can desensitize the user to it.

Continue...

- ▶ **Solution:** Interrupt whatever the user is doing with the message, using both sight and sound if possible. Communicate the message in clear, brief language that can be understood immediately, and provide information on how to remedy the situation, unless the cultural meaning of a non-lingual message is so strong that it cannot be mistaken (like a fire alarm). If the artifact shouldn't be used until the situation is dealt with, disable all actions until the message is acknowledged. Use different visual and aural cues for different classes of messages, so that a tense and distracted user has some basis for distinguishing between them. Bright colors, motion or flashing, and loud, strident, or shrill sounds all work to get a user's attention. Stop the alarm after acknowledgement, or at least let the user mute anything truly distracting.
- ▶ **Resulting Context:** Give the user an obvious way to acknowledge the message. If an audit trail is necessary or desirable, keep track of the messages over time, as with an [Interaction History](#).
- ▶ **Notes:** This is terribly overused. In truth, it's rarely the case that a user really must acknowledge a message before they can resume normal use. If this pattern is used with a non-critical message, a user's patience will quickly wear thin, and subsequent messages are at risk of being ignored (as in the boy who cried "Wolf") and bad things may happen. Use some form of [Status Display](#) for things which aren't critically important; don't shove them into the user's consciousness uninvited.
- ▶ There's got to be good reference material out there on this subject.

Mail System

- ▶ **Editable Collection**

- ▶ **Examples:**

- Shopping carts, both real and Web-based
- Email and voice-mail boxes
- Events in an electronic calendar

- ▶ **Context:** The user should build or modify an ordered set of things, possibly (but not necessarily) chosen from a larger set.

- ▶ **Problem:** How should the artifact indicate what the user is supposed to do with that collection?

- ▶ **Forces:**

- ▶ The user should know what the collection currently has in it.
- ▶ The user should be able to easily add and remove things to it, and reorder the collection if they wish.
- ▶ [unfinished]

Continue...

- ▶ **Solution:** Show the collection to the user, along with obvious ways to remove or change the position of each item. To add an item, make it eminently clear whether the user should obtain the item before or after the "add" command or gesture. Most of the time, this is clear from context -- if a user is shopping, obviously they have to pick the item out before they put it in their cart -- but other times it's not, so indicate it with a good metaphor, or good labeling (e.g. "Add..." with a subsequent dialog), or by the imposition of constraints (e.g. [Disabled Irrelevant Things](#)). If duplicate items in the collection aren't meaningful, then gently disallow them. In a visually-oriented artifact, direct manipulation is an excellent way of dealing with addition, removal, and ordering. Today's desktop GUIs offer drag-and-drop for this. If you use it, be sure to (1) offer a "dumping ground" for removed items, such as the familiar Mac trash can, if they don't get returned to their source; and (2) give some visual indication that you can do this, if there's no strong cultural indication to do so. (Software that use D&D as their primary means of interaction, such as drawing programs, have such cultural indications; others generally don't.)
- ▶ **Notes:** Obtaining an item before or after the "add" gesture -- this is part of the old "noun-verb vs. verb-noun" debate. [Any takers?] A good reason to use "noun-verb" is multiple selection: if you want to add or remove multiple items, the user needs to be able to pick them first, then do the operation on them (see [Actions for Multiple Objects](#)).